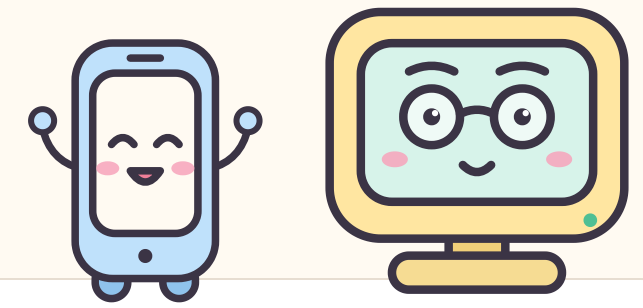




第 20 集 · 约 25 分钟

打包、安装与收尾

从“能跑”到“能交给别人用”

配套阅读：书的第 8 章



从零写出 LanShare

这一集你会学到

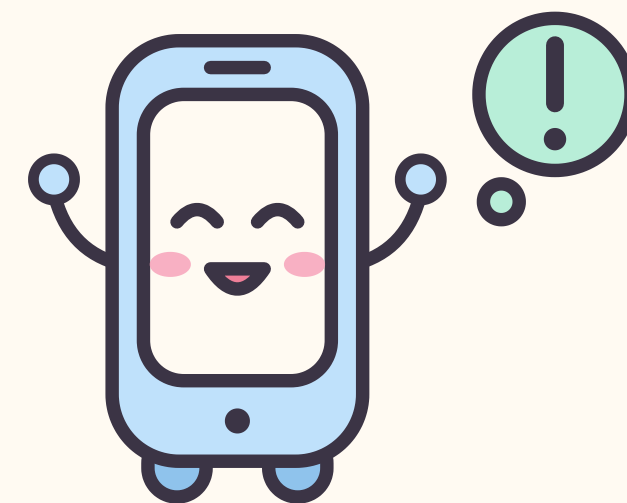
1. 一键构建脚本：编译、检查、测试、打包、冒烟测试一条龙
2. PowerShell 的两个坑：外部命令失败不会自动停下、中文脚本要存成带 BOM 的 UTF-8
3. 安装脚本：提权、开始菜单、设置里的卸载入口、防火墙
4. 卸载脚本：绝不误删用户的文件
5. 全课复盘：三轮安全审查，和这个程序诚实的边界



最后一段路

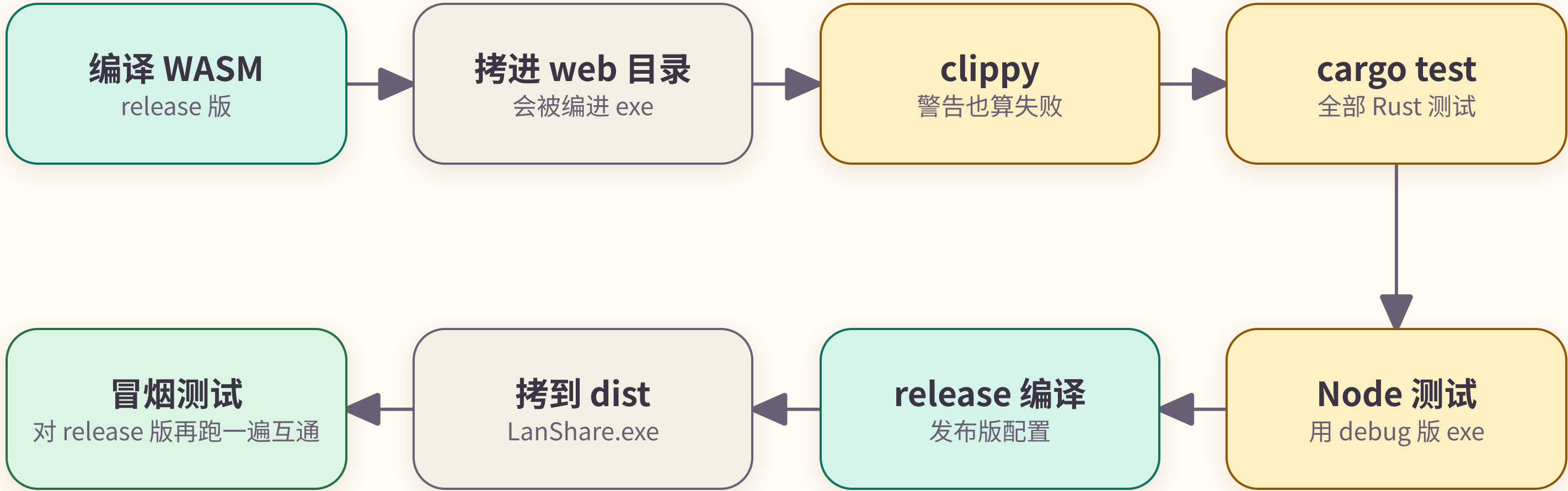
代码写完了，离一个能交给别人用的软件，还差什么？

一键构建并且自己把自己测一遍；像正规软件一样装进开始菜单；卸载时干干净净，还不能误删别人的东西。





build.ps1 做的事





build.ps1: 开头



build.ps1

第 4-23 行 • 共 23 行

```
4 # Keep this file ASCII-only: Windows PowerShell 5.1 reads BOM-less scripts in the local code page.
5 $ErrorActionPreference = "Stop"
6 Set-Location $PSScriptRoot
7
8 function Step($name, [scriptblock] $body) {
9     Write-Host "==> $name"
10    & $body
11    if ($LASTEXITCODE -ne 0) { throw "$name failed (exit $LASTEXITCODE)" }
12 }
13
14 # Release binaries embed the source paths of dependencies (for panic messages). They live under the
15 # cargo home, inside the user profile, so they would publish the user name: map them to "/cargo".
16 # CARGO_ENCODED_RUSTFLAGS (not RUSTFLAGS) so a path with spaces stays one argument.
17 $cargoHome = if ($env:CARGO_HOME) { $env:CARGO_HOME } else { Join-Path $env:USERPROFILE ".cargo" }
18 $env:CARGO_ENCODED_RUSTFLAGS = "--remap-path-prefix=$cargoHome=/cargo"
19
20 $targets = rustup target list --installed
21 if ($targets -notcontains "wasm32-unknown-unknown") {
22     throw "missing wasm target: run 'rustup target add wasm32-unknown-unknown' first"
23 }
```



build.ps1: 八个步骤



build.ps1

第 27-46 行 • 共 46 行

```
27
28 Step "clippy" { cargo clippy --workspace --all-targets -- -D warnings }
29 Step "cargo test" { cargo test --workspace }
30 Step "node tests (debug exe)" { node --test "web-tests/*.test.mjs" }
31 Step "release build" { cargo build -p lanshare --release }
32
33 New-Item -ItemType Directory -Force -Path dist | Out-Null
34 Copy-Item "target\release\LanShare.exe" "dist\LanShare.exe" -Force
35 $exe = (Resolve-Path "dist\LanShare.exe").Path
36
37 $env:LANSHARE_EXE = $exe
38 try {
39     Step "smoke test (release exe)" { node --test "web-tests/interop.test.mjs" }
40 } finally {
41     Remove-Item Env:\LANSHARE_EXE
42 }
43
44 $sizeKB = [math] ::Round((Get-Item $exe).Length / 1KB)
45 $wasmKB = [math] ::Round((Get-Item "crates\lanshare\web\lanshare.wasm").Length / 1KB, 1)
46 Write-Host "Done: dist\LanShare.exe ($sizeKB KB, wasm $wasmKB KB)"
```



一键构建

```
Windows PowerShell

PS D:\code\lanshare> powershell -NoProfile -ExecutionPolicy Bypass -File build.ps1
==> build wasm
..... (省略 25 行)
      Finished `release` profile [optimized] target(s) in 3.33s
==> clippy
..... (省略 142 行)
      Finished `dev` profile [unoptimized + debuginfo] target(s) in 40.84s
==> cargo test
..... (省略 119 行)
      Finished `test` profile [unoptimized + debuginfo] target(s) in 36.88s
..... (省略 39 行)
test result: ok. 35 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.03s
..... (省略 37 行)
test result: ok. 26 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.10s
..... (省略 26 行)
test result: ok. 21 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 1.92s
..... (省略 16 行)
test result: ok. 11 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.36s
..... (省略 6 行)
test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 1.21s
..... (省略 16 行)
```



安装要做的五件事

● 结束正在运行的旧版本

运行中的 exe 不能被覆盖

● 把 exe 拷进 Program Files

需要管理员权限，所以要弹一次 UAC

● 开始菜单里放一个快捷方式

固定到任务栏让用户自己右键，Windows 不给程序这个接口

● 登记到“设置 → 应用”

写注册表的卸载项，用户就能像卸载正规软件一样卸载它

● 防火墙放行入站连接，专用和公用网络都放行

家里的 WiFi 常被 Windows 归为公用网络



install.ps1: 准备

● ● ● install.ps1

第 23-42 行 • 共 42 行

```
23 }
24 $Exe      = Join-Path $InstallDir "LanShare.exe"
25 $Shortcut = Join-Path $StartMenu "$DisplayName.lnk"
26 $Log       = Join-Path $env:TEMP "LanShare-install.log"
27
28 function Test-Admin {
29     $identity = [Security.Principal.WindowsIdentity] ::GetCurrent()
30     (New-Object Security.Principal.WindowsPrincipal($identity)).IsInRole(
31         [Security.Principal.WindowsBuiltInRole] ::Administrator)
32 }
33
34 function Log($message) {
35     Write-Host $message
36     Add-Content -Path $Log -Value $message -Encoding UTF8
37 }
38
39 # 提权后的窗口一闪就关，从“设置”里调用时也没有控制台：出错一律弹窗（30 秒后自动关闭）
40 function Show-Failure($message) {
41     try { (New-Object -ComObject WScript.Shell).Popup($message, 30, $DisplayName, 0x10) | Out-Null } catch { }
42 }
```



install.ps1: 两个保险



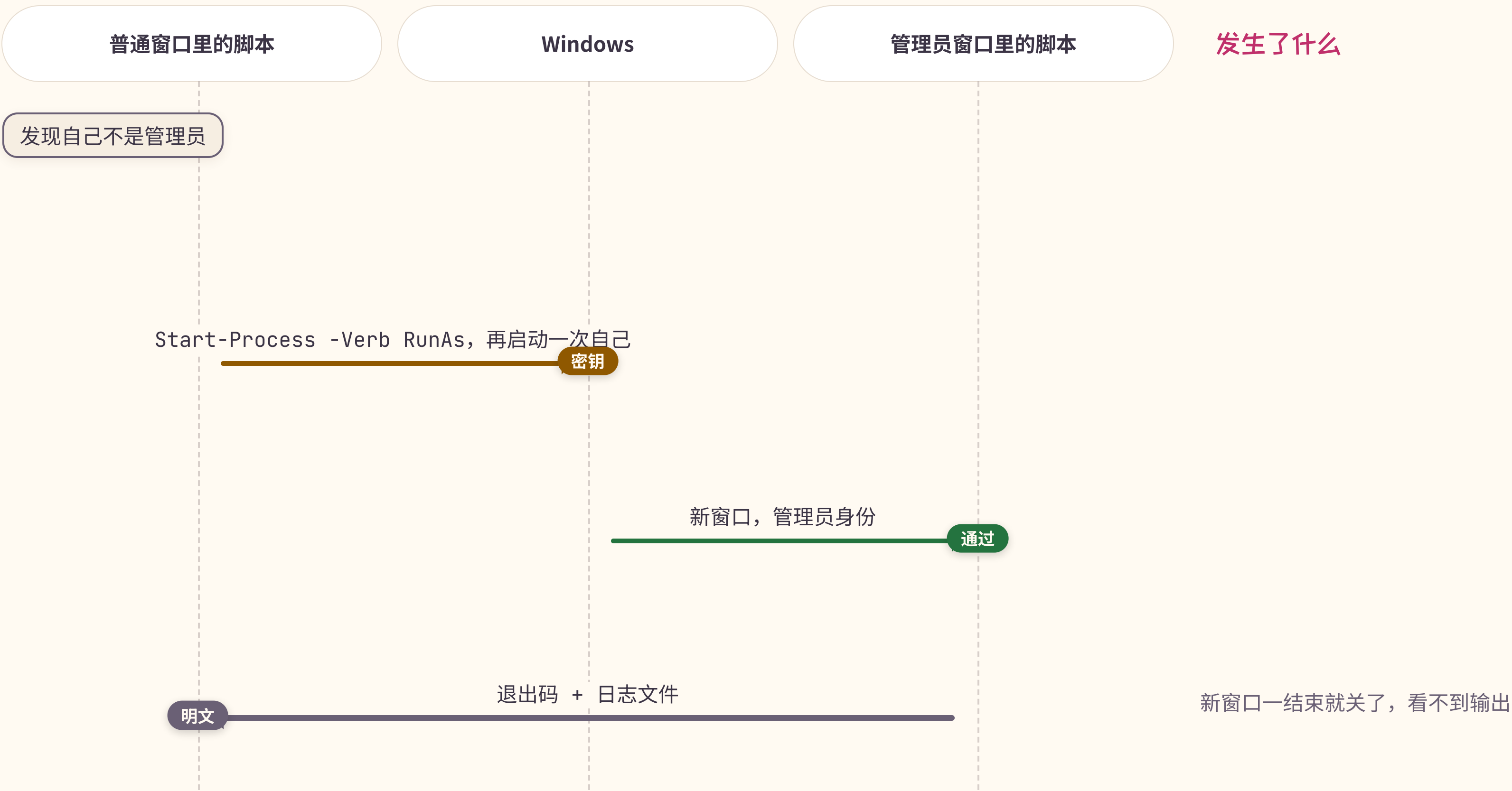
install.ps1

第 39–58 行 • 共 58 行

```
39 # 提权后的窗口一闪就关, 从“设置”里调用时也没有控制台: 出错一律弹窗 (30 秒后自动关闭)
40 function Show-Failure($message) {
41     try { (New-Object -ComObject WScript.Shell).Popup($message, 30, $DisplayName, 0x10) | Out-Null } catch { }
42 }
43
44 # 结束从安装目录启动的 LanShare (v1、v2 都算), 等它真正退出, exe 才能被覆盖
45 function Stop-Running {
46     $procs = @(Get-Process -Name $AppName -ErrorAction SilentlyContinue | Where-Object { $_.Path -eq $Exe })
47     foreach ($p in $procs) { Stop-Process -Id $p.Id -Force -ErrorAction SilentlyContinue }
48     foreach ($p in $procs) { [void] $p.WaitForExit(10000) }
49     if ($procs.Count -gt 0) { Log "已结束正在运行的 LanShare ($($procs.Count) 个进程) " }
50 }
51
52 # 杀毒软件可能还在扫刚释放的文件, 覆盖失败时重试几次
53 function Copy-WithRetry($from, $to) {
54     for ($i = 1; ; $i++) {
55         try { Copy-Item $from $to -Force; return }
56         catch { if ($i -ge 10) { throw }; Start-Sleep -Milliseconds 500 }
57     }
58 }
```



提权：自己用管理员身份再启动一次自己





install.ps1: 提权和复制

install.ps1

第 61-80 行 • 共 80 行

```
61      # 提权后在新窗口里执行, 结果写进日志, 再在这里显示出来
62      Remove-Item $Log -ErrorAction SilentlyContinue
63      $proc = Start-Process powershell.exe -Verb RunAs -Wait -PassThru `
64          -ArgumentList "-NoProfile -ExecutionPolicy Bypass -File `"$PSCmdPath`"
65      if (Test-Path $Log) { Get-Content $Log -Encoding UTF8 }
66      exit $proc.ExitCode
67  }
68
69  Remove-Item $Log -ErrorAction SilentlyContinue
70  try {
71      $source = Join-Path $PSScriptRoot "dist\LanShare.exe"
72      if (-not (Test-Path $source)) { throw "找不到 $source, 请先运行 build.ps1 打包" }
73      Stop-Running
74
75      New-Item -ItemType Directory -Force -Path $InstallDir | Out-Null
76      Copy-WithRetry $source $Exe
77      foreach ($name in "uninstall.ps1", "README.md") {
78          Copy-WithRetry (Join-Path $PSScriptRoot $name) (Join-Path $InstallDir $name)
79      }
80      Log "已复制到 $InstallDir"
```



install.ps1: 快捷方式和卸载入口

● ● ● install.ps1

第 90-109 行 • 共 109 行

```
90
91 $uninstallCmd = "powershell.exe -NoProfile -ExecutionPolicy Bypass -File `"$InstallDir\uninstall.ps1`"
92 if ($CurrentUser) { $uninstallCmd += " -CurrentUser" }
93 $sizeKB = [int] ((Get-ChildItem $InstallDir -File | Measure-Object -Property Length -Sum).Sum / 1KB)
94 New-Item -Path $UninstallKey -Force | Out-Null
95 $strings = [ordered] @{
96     DisplayName      = $DisplayName
97     DisplayVersion    = $Version
98     Publisher         = $AppName
99     DisplayIcon       = "$Exe,0"
100     InstallLocation   = $InstallDir
101     UninstallString    = $uninstallCmd
102 }
103 foreach ($key in $strings.Keys) {
104     New-ItemProperty -Path $UninstallKey -Name $key -Value $strings[$key] -PropertyType String -Force | Out-Null
105 }
106 foreach ($pair in @(@"NoModify", 1), @"NoRepair", 1), @"EstimatedSize", $sizeKB)) {
107     New-ItemProperty -Path $UninstallKey -Name $pair[0] -Value $pair[1] -PropertyType DWord -Force | Out-Null
108 }
109 Log "已登记到 设置 → 应用"
```



install.ps1: 防火墙

install.ps1

第 107-126 行 • 共 126 行

```
107         New-ItemProperty -Path $UninstallKey -Name $pair[0] -Value $pair[1] -PropertyType DWord -Force | Out-Null
108     }
109     Log "已登记到 设置 → 应用"
110
111     if (-not $CurrentUser) {
112         Get-NetFirewallRule -Group $AppName -ErrorAction SilentlyContinue | Remove-NetFirewallRule
113         # 所有网络类型都放行: Windows 常把家里的 WiFi 默认归为“公用网络”，只放行“专用”会让手机连不上。
114         # 而且只要程序已有任何一条规则, Windows 就不再弹窗询问, 连接会被静默拦截。
115         New-NetFirewallRule -DisplayName $DisplayName -Group $AppName -Direction Inbound -Action Allow `
116             -Program $Exe -Protocol TCP -Profile Any | Out-Null
117         Log "防火墙: 已放行入站连接 (专用和公用网络) "
118     }
119     Log "安装完成: $DisplayName $Version (从开始菜单或任务栏启动) "
120     exit 0
121 } catch {
122     $message = "安装失败: $($_.Exception.Message)"
123     Log $message
124     Show-Failure $message
125     exit 1
126 }
```



uninstall.ps1: 先想清楚删什么

● ● ● uninstall.ps1

第 10-29 行 • 共 29 行

```
10 if ($CurrentUser) {
11     $DefaultDir = Join-Path $env:LOCALAPPDATA "Programs\$AppName"
12     $StartMenu = Join-Path $env:APPDATA "Microsoft\Windows\Start Menu\Programs"
13     $UninstallKey = "HKCU:\Software\Microsoft\Windows\CurrentVersion\Uninstall\$AppName"
14 } else {
15     $DefaultDir = Join-Path $env:ProgramFiles $AppName
16     $StartMenu = Join-Path $env:ProgramData "Microsoft\Windows\Start Menu\Programs"
17     $UninstallKey = "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\$AppName"
18 }
19 # 安装目录以安装时登记的 InstallLocation 为准，不按脚本自己所在的位置猜：
20 # 这个脚本可能被拷到别处运行（比如被传进了同样叫 LanShare 的共享文件夹，独立安全审查 M-1）。
21 $InstallDir = $DefaultDir
22 $registered = (Get-ItemProperty -Path $UninstallKey -ErrorAction SilentlyContinue).InstallLocation
23 if ($registered -and (Split-Path $registered -Leaf) -eq $AppName) { $InstallDir = $registered }
24 # 安装时放进安装目录的全部文件：卸载只删这些，然后只在目录空了时删目录
25 $InstalledFiles = "LanShare.exe", "uninstall.ps1", "README.md"
26 $Exe = Join-Path $InstallDir "LanShare.exe"
27 $Shortcut = Join-Path $StartMenu "$DisplayName.lnk"
28 $Pinned = Join-Path $env:APPDATA "Microsoft\Internet Explorer\Quick Launch\User Pinned\TaskBar\$DisplayName.lnk"
29 $Log = Join-Path $env:TEMP "LanShare-uninstall.log"
```



uninstall.ps1: 只删自己的东西

uninstall.ps1

第 95-114 行 • 共 114 行

```
95     }
96     $kept = Remove-IfEmpty $InstallDir
97     # 日志和单实例记录, 同样只删认识的文件
98     $DataDir = Join-Path $env:LOCALAPPDATA $AppName
99     foreach ($name in "lanshare.log", "instance.json", "instance.json.tmp") {
100         Remove-Item -LiteralPath (Join-Path $DataDir $name) -Force -ErrorAction SilentlyContinue
101     }
102     [void] (Remove-IfEmpty $DataDir)
103
104     # 注册表项最后删: 前面任何一步失败, “设置 → 应用”里还留着入口, 可以再点一次卸载
105     Remove-Item $UninstallKey -Recurse -Force -ErrorAction SilentlyContinue
106     if ($kept) { Log "安装目录里还有不是安装程序放进去的文件, 已保留: $InstallDir" }
107     Log "已卸载 $DisplayName。共享文件夹(下载\LanShare)里的文件没有动。"
108     exit 0
109 } catch {
110     $message = "卸载失败: $($_.Exception.Message)"
111     Log $message
112     Show-Failure $message
113     exit 1
114 }
```



装好以后

● 运行 install.ps1，点一次 UAC 的“是”

● 开始菜单里出现“局域网快传 LanShare”

● 想放到任务栏：右键 → 固定到任务栏

这一步留给用户自己点，程序不替用户做

● 卸载：设置 → 应用 → 局域网快传 LanShare → 卸载



回头看：三轮安全审查

轮次	发现的问题（节选）	我们的处理
第一轮：独立审查	口令可以被长期在线猜测，30 天约 58% 的概率猜中	累计 30 次没完成的尝试就暂停口令登录，只能在电脑上重新开启
第一轮：独立审查	不需要密钥就能让服务端把 1 MiB 请求体收进内存，300 个连接占 526 MiB	先验请求头再读正文；连接有上限，空闲 60 秒断开
第一轮：独立审查	卸载脚本可能把同名的共享文件夹整个删掉	以注册表登记的位置为准，只删 3 个文件
第一轮：独立审查	Rust 的 fs rename 在 Windows 上会覆盖同名文件	改用不覆盖的改名
复验	第一版修复带来新问题：连接上限不分来源，一台机器就能占满	自己接收连接，每个 IP 单独限额，本机单独一个池子
第三轮复验	握手接口读正文没有时限：每 25 秒发一个字节，连接能一直挂着	握手正文限量 4 KiB、限时 10 秒





诚实的边界： 它不防什么

主动攻击

页面通过 http 下发，有人搭恶意热点、篡改首次加载的页面，就能拿到密钥

流量特征

传了多大的文件、什么时候传，窃听者仍然看得到

拿到二维码或口令的人

他就是可信用户；想让所有设备失效：退出程序再打开

落盘不加密

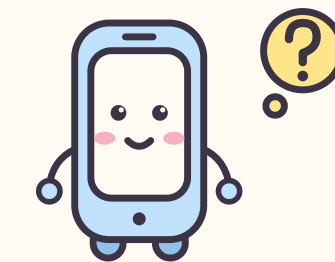
加密只保护传输过程



整个项目



```
lanshare/  
├── crates/ 47 个文件  
├── web-tests/ 4 个文件  
├── build.ps1 这一集写的  
├── Cargo.toml  
├── install.ps1 这一集写的  
└── uninstall.ps1 这一集写的
```



选择题

build.ps1 第一行就设置了“出错就停下”，为什么每一步还要检查 LASTEXITCODE?

- A 为了打印更好看的错误信息
- B 因为“出错就停下”只管 PowerShell 自己的命令，cargo、node 这些外部程序失败时，脚本会照样往下走
- C 因为 cargo 不会返回退出码



选择题

build.ps1 第一行就设置了“出错就停下”，为什么每一步还要检查 LASTEXITCODE?

A 为了打印更好看的错误信息

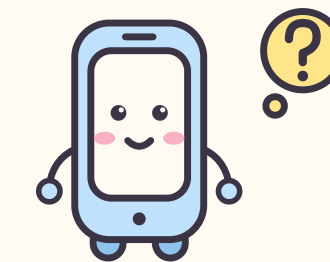
这不是主要原因。不检查的话，失败根本不会被发现。

C 因为 cargo 不会返回退出码

cargo 会返回退出码，失败时不是 0。问题在于 PowerShell 不会自动检查它。

B 因为“出错就停下”只管 PowerShell 自己的命令，cargo、node 这些外部程序失败时，脚本会照样往下走

外部程序的失败只会体现在退出码上，不检查退出码，测试失败了也会继续打包，交出一个没测过的 exe。



选择题

卸载脚本为什么不按“脚本自己所在的目录”去删安装目录？

- A 因为算脚本所在目录的代码太复杂
- B 因为脚本可能被拷到别处运行，比如同样叫 LanShare 的共享文件夹，那样会删掉用户的文件
- C 因为 PowerShell 不能删除自己所在的目录



选择题

卸载脚本为什么不按“脚本自己所在的目录”去删安装目录？

A 因为算脚本所在目录的代码太复杂

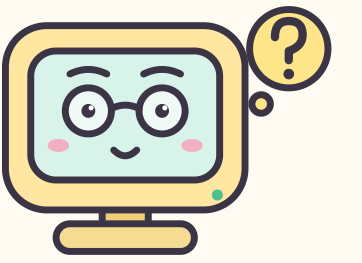
一行就能算出来，这不是原因。

C 因为 PowerShell 不能删除自己所在的目录

先把当前目录挪开就能删，脚本里也确实这样做了，但这不是不按脚本位置删的原因。

B 因为脚本可能被拷到别处运行，比如同样叫 LanShare 的共享文件夹，那样会删掉用户的文件

这是安全审查发现的问题。现在以注册表里登记的安装位置为准，而且只删安装时放进去的 3 个文件。



学完整门课，如果让你给 LanShare 加一个新功能，比如“网页上可以删除文件”，你要改哪些地方？要考虑哪些安全问题？



想一想

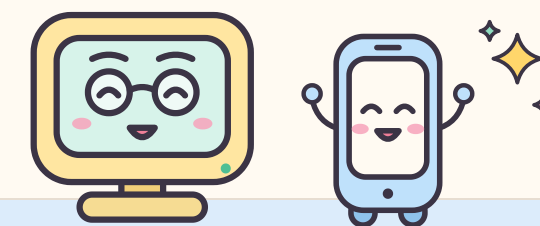
学完整门课，如果让你给 LanShare 加一个新功能，比如“网页上可以删除文件”，你要改哪些地方？要考虑哪些安全问题？

协议上要加一个新的加密接口，或者在 RPC 里加一个方法；服务端要写处理函数，用第 9 集 resolve 的三道防线，防止删到共享文件夹外面；要想清楚谁能删：任何一台连上的设备都能删，可能误删别人传的文件；还要写测试，包括路径穿越的攻击用例；最后网页上加按钮和确认提示。



正课里，我们写了什么

- ✓ 协议核心：ChaCha20-Poly1305 加密信道、HKDF 和 HMAC 握手、SPAKE2 口令登录、防重放、文件名清理
- ✓ WebAssembly：同一份 Rust 代码在浏览器里运行，不依赖 wasm-bindgen
- ✓ 服务端：axum 和 tokio，连接管理、配对、加密接口、分块传输，每一道防线都有测试
- ✓ 桌面外壳：单实例、主界面窗口、托盘
- ✓ 网页：协议客户端和界面
- ✓ 打包、安装、卸载：一键构建并自测，像正规软件一样安装，卸载绝不误删



下一集

全课完结，感谢收看